

Centro de Distribuição Automatizado: Simulação Concorrente com Raylib

Artur Moraes, Helenna Massarra Paes, Pedro Calil Raposo

¹Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP)
Ciência da Computação e Engenharia de Software
Disciplina: Programação Concorrente — 2026/1

{artur.moraes, helenna.massarra, pedro.raposo}@idp.edu.br

Resumo. Este trabalho implementa a simulação de um centro de distribuição automatizado composto por robôs coletores, robôs entregadores e uma esteira transportadora. A implementação em C usa *pthread*s para modelar cada entidade como uma *thread* independente e *mutex*es por recurso para coordenar o acesso a células do mapa, esteira e filas das estações. A interface gráfica usa Raylib em substituição ao *ncurses*, habilitando o bônus do enunciado. Três cenários estáticos e uma configuração dinâmica validam o sistema sob diferentes cargas.

1. Organização das Threads

O programa cria $N + M + 3$ *threads* POSIX [2] ao total, onde N é o número de robôs coletores e M o de entregadores. A Tabela 1 resume as *threads* e seus argumentos.

Tabela 1. Threads do sistema

Thread	Função	Qtd.	Argumento
Gerador de pacotes	<code>thread_gerador</code>	1	<code>Simulacao *</code>
Esteira	<code>thread_esteira</code>	1	<code>Simulacao *</code>
Robô coletor	<code>thread_coletor</code>	N	<code>ArgRobo *</code>
Robô entregador	<code>thread_entregador</code>	M	<code>ArgRobo *</code>
Interface Raylib	<code>thread_interface</code>	1	<code>Simulacao *</code>

Todas são criadas em `main.c` com `pthread_create` e aguardadas com `pthread_join`. A janela Raylib é aberta *dentro* de `thread_interface` via `InitWindow`, pois a biblioteca exige que o contexto OpenGL seja criado e usado pela mesma *thread*. O encerramento é cooperativo: quando a flag `sim.rodando` vai a zero, todos os laços terminam naturalmente e os `pthread_join` retornam.

2. Recursos Compartilhados

A Tabela 2 lista os recursos compartilhados e seus acessores.

3. Seções Críticas Identificadas

Movimentação de robôs (`mapa.c`). Dois robôs podem tentar ocupar a mesma célula simultaneamente. `mapa_tenta_mover` adquire `Mapa.mutex`, verifica se o destino

Tabela 2. Recursos compartilhados do sistema

Recurso	Tipo	Acessado por
Mapa.celulas[] .ocupante	grade de células	todos os robôs (r/w de posição) e interface (leitura)
Esteira.posicoes[]	buffer linear	coletores (inserção), esteira (avanço), entregadores (retirada), interface
Estacao (cabeca, cauda, quantidade)	fila encadeada	gerador (enfileira) e coletores (desenfileira)
Estatisticas	contadores inteiros	gerador e entregadores (escrita); interface (leitura)
Simulacao.rodando	flag de parada	qualquer <i>thread</i> pode escrever; todas leem

está livre, atualiza origem e destino e libera o *mutex* em um único *lock*, garantindo exclusão mútua.

Esteira (*esteira.c*). O vetor `posicoes[]` é acessado por três *threads*: inserção em `[0]`, avanço de `[i-1]` para `[i]` e retirada em `[tamanho-1]`. Todas as operações adquirem `Esteira.mutex` antes de qualquer acesso.

Filas de estações (*estacao.c*). Gerador e múltiplos coletores disputam a mesma estação: enfileirar e desenfileirar adquirem `Estacao.mutex`, garantindo que lista encadeada e contador sejam modificados atomicamente.

Estatísticas (*simulacao.c*). `stats_inc_entregues` verifica a condição de término *sob o mesmo lock*: atingida a meta, chama `simulacao_parar` ainda na seção crítica, evitando parada duplicada por corrida.

4. Mecanismos de Sincronização

`pthread_mutex_t` — exclusão mútua por recurso. Cada recurso possui seu próprio *mutex* (`Mapa.mutex`, `Esteira.mutex`, `Estacao.mutex` por estação, `Estatisticas.mutex`). A granularidade fina permite que *threads* que acessam recursos distintos avancem em paralelo. Nenhuma *thread* adquire dois *locks* simultaneamente, eliminando o risco de *deadlock* por espera circular.

`volatile sig_atomic_t rodando` — flag de término. `volatile` impede que o compilador cache o valor em registrador. Formalmente, `sig_atomic_t` garante atomicidade apenas frente a *signal handlers*, não entre *threads*; a corretude aqui apoia-se em cargas e armazenamentos alinhados serem atômicos em x86-64 e na flag transitar uma única vez, de 1 para 0. O pior caso é uma *thread* executar um *tick* extra após a parada — aceitável nesta simulação. Uma alternativa estritamente portátil seria `_Atomic int` de `stdatomic.h` (C11).

Espera ativa com `dormir_ms`. Nos pontos de espera (coletor aguardando espaço na esteira; entregador aguardando pacote na saída), as *threads* dormem `passo_ms` e tentam novamente. Como o intervalo entre tentativas é limitado por `passo_ms` — o período que dita o ritmo da simulação —, o custo do *polling* é desprezível: no máximo uma verificação por *tick*, sem consumo de CPU entre elas. `pthread_cond_t` reduziria a latência de notificação, mas ela já é mascarada pelo próprio passo da simulação.

5. Decisões de Projeto

Interface gráfica com Raylib (bônus +10%). O enunciado prevê *ncurses* como padrão e oferece bônus para grupos que substituam integralmente a interface por Raylib [1]. Este projeto implementa a interface com Raylib: o mapa é renderizado como grade de células coloridas e o painel lateral exibe esteira, contadores e progresso em tempo real (Figura 1). A *thread* de interface captura um *snapshot* do estado sob *mutex* e renderiza em seguida, sem manter recursos travados durante o desenho.

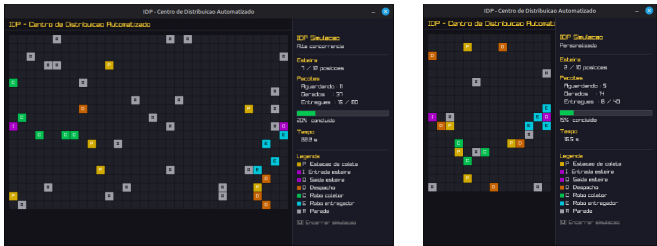


Figura 1. Interface Raylib nos cenários Alta Concorrência (esq.) e Personalizado/dinâmico (dir.).

Mutex por recurso em vez de mutex global. Um *mutex* global criaria gargalo: a interface bloquearia todos os robôs durante a renderização. Com *mutexes* por recurso, cada robô bloqueia apenas a célula que tenta ocupar, e a interface segura `Mapa.mutex` apenas durante uma cópia de buffer.

BFS com leitura *lock-free*. Cada passo recalcula o BFS [3] lendo *ocupante* sem *mutex*. Dados desatualizados apenas fazem o robô tentar um passo que falha em `mapa_tenta_mover` (atômico); o próximo ciclo recalcula. Isso elimina contenção no *mutex* do mapa durante o planejamento.

Heurística gulosa para escolha de destino. Coletores escolhem a estação *não-vazia mais próxima* (distância de Manhattan); entregadores, o *despacho mais próximo*. Não é ótimo globalmente, mas distribui robôs entre estações/despachos sem comunicação entre *threads*.

Lógica afastar_da_entrada. Coletores ociosos junto à entrada da esteira poderiam bloquear o último coletor carregado. A função move o coletor ocioso para a célula vizinha mais distante da entrada, liberando o gargalo.

6. Guia de Utilização

A interface exige Raylib 5.5; o README do repositório traz a instalação a partir do fonte no Ubuntu 24.04. Alternativamente, o Dockerfile incluso constrói imagem pronta (`podman build -t idp_sim .`), executável montando `/tmp/.X11-unix` e exportando `DISPLAY`. Com a biblioteca disponível:

```
make # compila ./idp_sim
./idp_sim 1 # Cenário Simples (18x12, 20 pacotes)
./idp_sim 2 # Cenário Intermediario (24x16, 40 pacotes)
./idp_sim 3 # Alta Concorrência (32x20, 80 pacotes)
./idp_sim 0 # Configuracao dinamica (le parametros)
make teste teste-bfs teste-stress # testes automatizados
# tecla Q encerra a simulacao antecipadamente
```

7. Resultados

Os três cenários estáticos e a configuração dinâmica foram executados em Ubuntu 24.04; a Tabela 3 resume os resultados. O cenário dinâmico usou parâmetros não-padrão (5 estações, 5 despachos, 7 obstáculos, esteira de 10, *tick* de 250 ms), exercitando o leitor de configuração.

Tabela 3. Resultados observados por cenário

Cenário	Mapa	Pacotes	Colet.	Entreg.	Entregues	Tempo (s)
1 — Simples	18 × 12	20	2	1	20/20	76,2
2 — Intermediário	24 × 16	40	3	2	40/40	83,8
3 — Alta Concorrência	32 × 20	80	4	4	80/80	92,5
0 — Dinâmico	14 × 18	40	3	4	40/40	65,0

Em todas as execuções, a totalidade dos pacotes foi entregue e a simulação encerrou de forma limpa, com todos os `pthread_join` retornando. Não foram observados *deadlocks* nem corrupção de estado: nenhum robô ocupou célula já ocupada, nenhum pacote foi perdido ou duplicado na esteira, e as estatísticas fecharam consistentes com o total gerado.

No Cenário 3 observou-se maior disputa pelas células adjacentes à entrada da esteira; a lógica `afastar_da_entrada` (Seção 5) mostrou-se necessária para evitar bloqueio do último coletor carregado. Os testes automatizados passaram em todas as execuções, incluindo repetições consecutivas para exercitar intercalações distintas de *threads*.

8. Conclusão

Este trabalho apresentou a implementação de um centro de distribuição automatizado como um sistema concorrente em C, com cada entidade executando como uma *thread* POSIX independente. A estratégia de um *mutex* por recurso, combinada a seções críticas curtas e à ausência de aquisição aninhada de *locks*, garantiu exclusão mútua sem *deadlocks* e com paralelismo real entre *threads* que operam sobre recursos distintos.

Os experimentos com os três cenários estáticos e a configuração dinâmica confirmaram o comportamento esperado: todos os pacotes foram entregues, o encerramento cooperativo funcionou e o sistema manteve-se estável mesmo sob alta concorrência. Como trabalhos futuros, destacam-se a substituição da flag de término por `stdatomic.h` (portabilidade estrita), o uso de variáveis de condição caso a granularidade temporal da simulação seja reduzida, e políticas de roteamento cooperativas para reduzir a contenção nos gargalos do mapa.

Referências

- [1] Santamaria, R. (2013–2025). *raylib*. <https://www.raylib.com>.
- [2] IEEE and The Open Group (2018). *IEEE Std 1003.1-2017 — Portable Operating System Interface (POSIX)*. IEEE, New York.
- [3] Cormen, T. H., Leiserson, C. E., Rivest, R. L., and Stein, C. (2009). *Introduction to Algorithms*, 3rd ed. MIT Press.